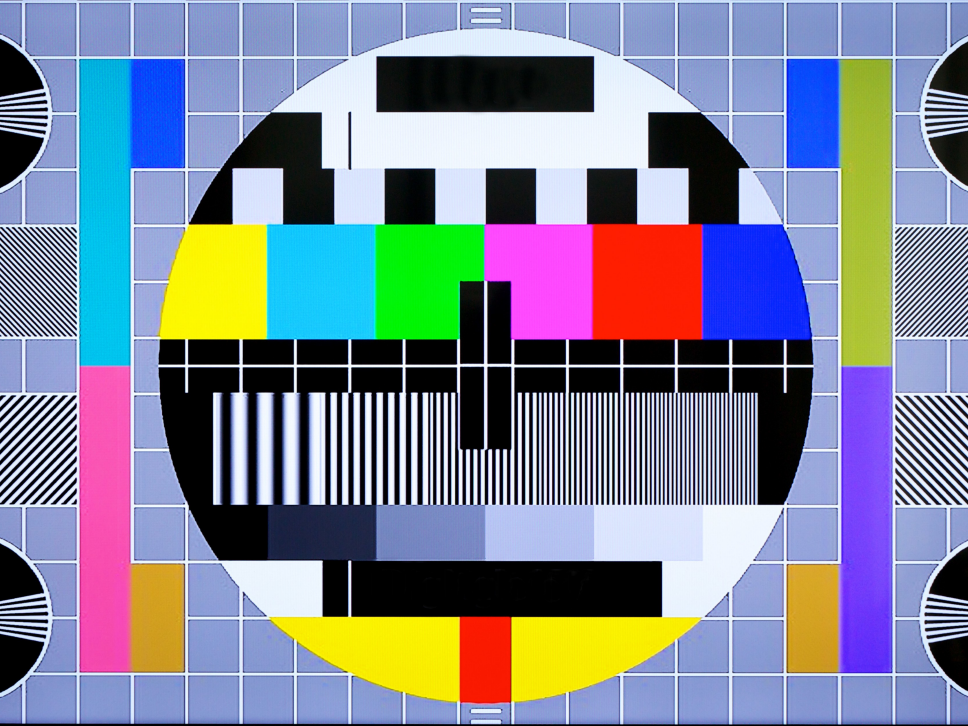




Minimal Comparison of Octagonal Abstract Domains

Kenny Ballou¹
Elena Sherman²

¹California State University San Marcos

²Boise State University

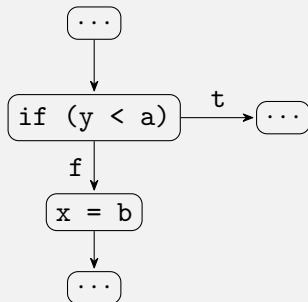
October 2026

Outline

- ① Background
- ② Approach
- ③ Experimental Results
- ④ Discussion

Static analysis computes information about programs

```
1  // ...  
2  if (y >= a) {  
3      x = b;  
4      // ...  
5  } else {  
6      // ...  
7  }
```

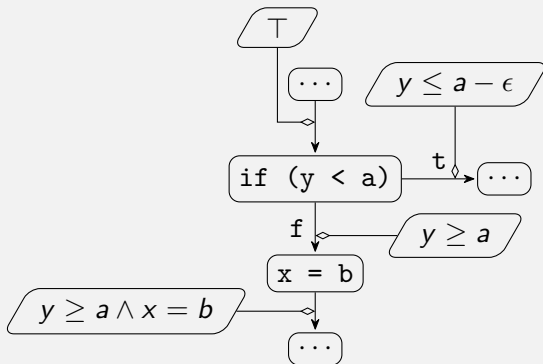


Static analysis computes information about programs

```

1  // ...
2  if (y >= a) {
3      x = b;
4      // ...
5  } else {
6      // ...
7  }

```

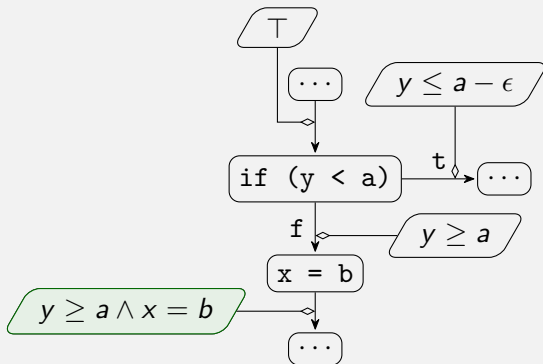


Static analysis computes information about programs

```

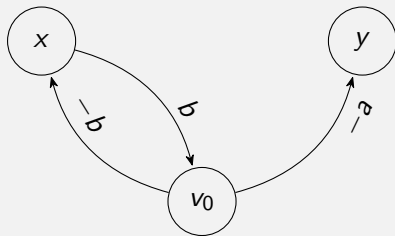
1 // ...
2 if (y >= a) {
3   x = b;
4   // ...
5 } else {
6   // ...
7 }

```



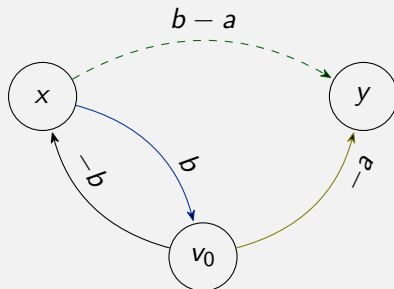
Zones encode unit differences between two variables

$$\begin{array}{r}
 x - v_0 \leq b \\
 v_0 - x \leq -b \\
 v_0 - y \leq -a \\
 \hline
 x \leq b \\
 x \geq b \\
 y \geq a
 \end{array}$$



Transitive closure makes implicit inferred invariants explicit

$$\begin{array}{l}
 x \leq b \\
 x \geq b \\
 y \geq a \\
 \hline
 x - y \leq b - a
 \end{array}$$



Octagons encode unit sums between two variables

$$\begin{array}{r}
 x \leq b \\
 x \geq b \\
 y \geq a \\
 \hline
 x^+ - x^- \leq 2b \\
 x^- - x^+ \leq -2b \\
 y^- - y^+ \leq -2a
 \end{array}$$

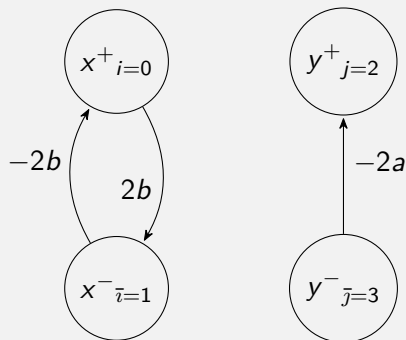


Figure: Graphical representation of the running example of octagonal constraints

Infinitely many Octagons can represent the same bounded region

$$\begin{array}{rcl}
 y & \geq & a \\
 x & \leq & b \\
 x & \geq & b \\
 \hline
 x - y & \leq & b - a \\
 -x - y & \leq & -a - b
 \end{array}$$

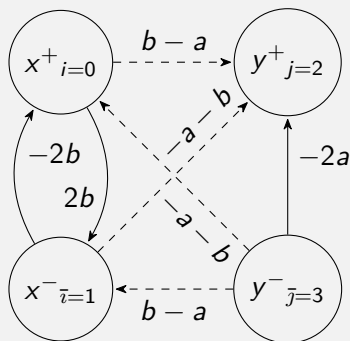


Figure: Graphical representation of the running example of octagonal constraints

Transitive closure introduces *spurious constraints*

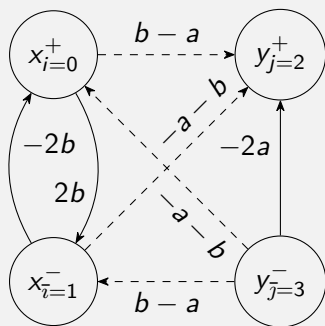


Figure: Fully closed Octagonal state after interpreting $x = b$

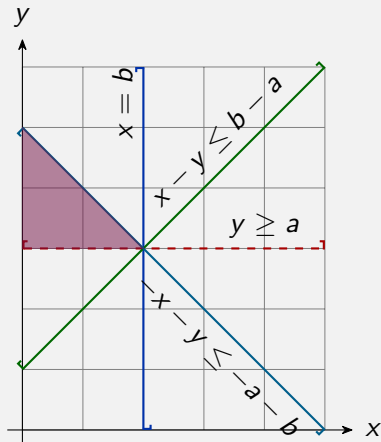


Figure: Projection of the bounded region represented by the current octagonal invariants

Spurious constraints are inferable, redundant constraints

Definition

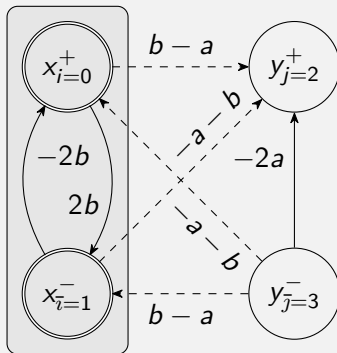
Octagonal Spurious Constraints

$$i \neq j \wedge i \oplus j \neq 1 \quad (1)$$

$$M_{i,j} \geq \frac{M_{i,\bar{i}} + M_{\bar{j},j}}{2} \quad (2)$$

Leveraging incremental nature of DFA, we can minimize the comparison

3 `x = b;`



Incremental change information provides a selection criterion

Algorithm Minimum Changed Variable Selection Algorithm

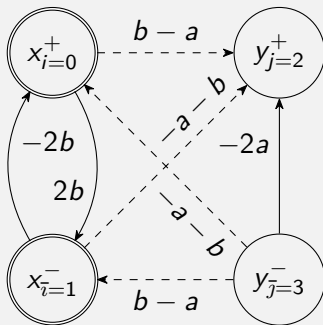
Require: M is a strongly closed Octagonal difference bounded matrix (DBM) and de be the set of changed vertices within M .

```

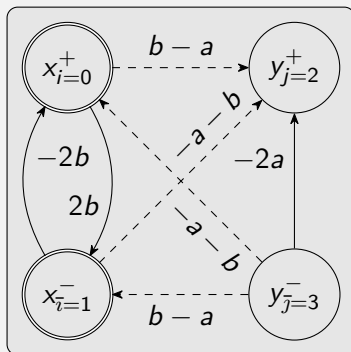
1:  $dv \leftarrow \{\}$ 
2: for  $(i, j) \in de$  do
3:   if  $i \equiv j \vee \text{IsIntervalValued}(i, j)$  then
4:      $dv \leftarrow dv \cup \{i, j\}$ 
5:   else
6:      $dv \leftarrow dv \cup \{i\}$ 
7:   end if
8: end for
9:  $\Delta\text{vars} \leftarrow dv$ 
10: for  $v \in dv$  do
11:    $\Delta\text{vars} \leftarrow \Delta\text{vars} \cup \{i \mid i \in \{0 \dots 2N\} \wedge M_{v,i} < +\infty\}$  {Select forward reachable variables}
12:    $\Delta\text{vars} \leftarrow \Delta\text{vars} \cup \{i \mid i \in \{0 \dots 2N\} \wedge M_{i,v} < +\infty\}$  {Select backward reachable variables}
13: end for
14:
15: return  $\Delta\text{vars}$ 

```

Example of the selection criterion



Example of the selection criterion



✗ All variables are selected!

Reduction Algorithm

Algorithm “W0Z”-Reduction Algorithm for Octagons

Require: $M \equiv \mathcal{C}(M)$: M is a fully closed Octagon DBM

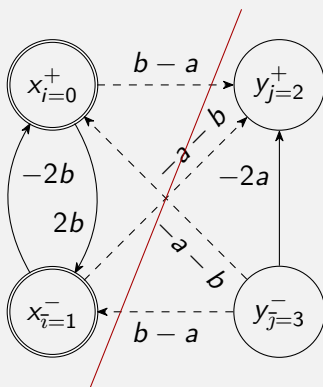
Ensure: $(\mathcal{C} \circ \mathcal{R})(M) \equiv M$: The closure of the reduction is equivalent to M .

```

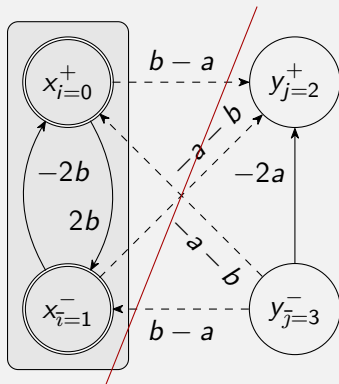
1: for all  $i \in \{0, \dots, 2N\}$  do
2:   for all  $j \in \{0, \dots, 2N\}$  do
3:     if  $i \equiv j \vee \text{IsIntervalValued}(i, j)$  then
4:       continue
5:     end if
6:     if  $M_{i,j} \geq \frac{M_{i,\bar{i}} + M_{\bar{j},j}}{2}$  then
7:        $M_{i,j} \leftarrow +\infty$ 
8:     end if
9:   end for
10: end for

```

Example of variable selection criterion using reduction



Example of variable selection criterion using reduction



✓ Only changed variables are selected!

Experimental Evaluation

Experimental Evaluation

- Compared Octagons to *comparable* domains: Intervals and Zones
- Compared Octagons to *incomparable* domain: Symbolic Predicates

Experimental Evaluation

Experimental Evaluation

- Compared Octagons to *comparable* domains: Intervals and Zones
- Compared Octagons to *incomparable* domain: Symbolic Predicates

Subject Programs

- 192 Java methods selected from previous research
- 6930 computed and compared invariants

Logical entailment classifies precision into 2/5 buckets

Comparable

- $\equiv$ Equivalent Precision
- $\prec$ Less Precise

Incomparable

- $\equiv$ Equivalent Precision
- $\prec$ Less Precise
- $\succ$ More Precise
- $\prec \succ$ Incomparable
- ? (SMT) Unknown

Logical entailment SMT Queries

Logical Entailment

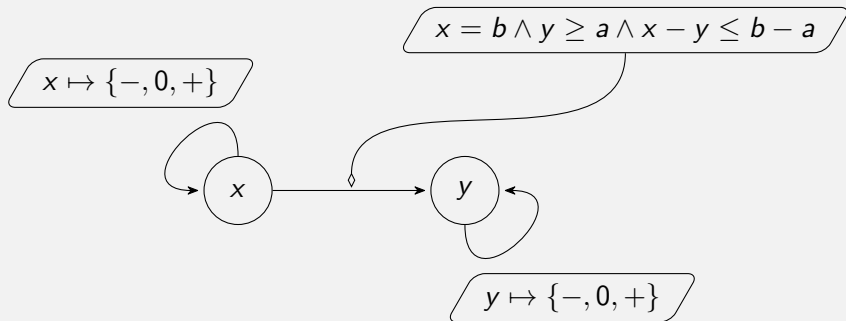
```
1 (push)
2 (forall ((x Int) (y Int))
3     (assert (=> (and (= x b) (>= y a))
4                 (and (= x b) (>= y a)
5                     (<= (- x y) (- b a))))))
6 (check-sat)
7 (pop)
8 (push)
9 (forall ((x Int) (y Int))
10    (assert (=> (and (= x b) (>= y a)
11                    (<= (- x y) (- b a)))
12            (and (= x b) (>= y a)))))
13 (check-sat)
14 (pop)
```

Summary comparison between Octagons, Intervals and Zones

Table: Invariant Comparisons between Intervals and Zones vs Octagons

Technique	$I \equiv O$	$I \prec O$	$Z \equiv O$	$Z \prec O$
Full Comparison	1523	5407	5138	1792
MN	3894	3036	6183	747
MN + Reduction	4439	2491	6527	403

Symbolic predicates¹ augment Predicate Domain elements with precise, symbolic invariants



¹Sherman and Dwyer, "Exploiting Domain and Program Structure to Synthesize Efficient and Precise Data Flow Analyses (T)".

Summary comparison between Octagons and Symbolic Predicates

Table: Invariant Comparisons between Intervals and Zones vs Octagons

Technique	$O \equiv P$	$O \prec P$	$O \succ P$	$O \prec \succ P$	$O ? P$
Full	1179	3082	499	2159	11
MN + Reduc	3768	2514	280	368	0

Discussion

- Reduction and Minimized Comparison offer a more granular comparison between comparable and incomparable abstract domains
- This work offers several techniques for comparing different numerical abstract domains

Future Work

- Leverage change information to improve analysis performance
- Combine our reduction algorithm with Larsen et al.'s^a to improve state reduction
- Combine these reduction techniques with other metric-based^b techniques for abstract domain comparison

^aLarsen et al., “Efficient Verification of Real-Time Systems: Compact Data Structure and State-Space Reduction”.

^bCasso et al., “Computing Abstract Distances in Logic Programs”.

Thank you

Questions?

The work reported here was supported by the U.S. National Science Foundation under award CCF-19-42044.

References I

- [1] Ignacio Casso et al. “Computing Abstract Distances in Logic Programs”. In: *Lecture Notes in Computer Science* (2020), pp. 57–72. ISSN: 1611-3349. DOI: 10.1007/978-3-030-45260-5_4. URL: http://dx.doi.org/10.1007/978-3-030-45260-5_4.
- [2] K.G. Larsen et al. “Efficient Verification of Real-Time Systems: Compact Data Structure and State-Space Reduction”. In: *Proceedings Real-Time Systems Symposium*. IEEE Comput. Soc, 1997, pp. 14–24. ISBN: 081868268X. DOI: 10.1109/real.1997.641265. URL: <http://dx.doi.org/10.1109/REAL.1997.641265>.

References II

- [3] Elena Sherman and Matthew B. Dwyer. “Exploiting Domain and Program Structure to Synthesize Efficient and Precise Data Flow Analyses (T)”. In: *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (Nov. 2015). DOI: 10.1109/ase.2015.41. URL: <http://dx.doi.org/10.1109/ASE.2015.41>.

Octagon Encoding Rules

$$\begin{aligned}
 x - y \leq c &\rightsquigarrow x^+ - y^+ \leq c \wedge y^- - x^- \leq c \\
 x + y \leq c &\rightsquigarrow x^+ - y^- \leq c \wedge y^+ - x^- \leq c \\
 -x - y \leq c &\rightsquigarrow x^- - y^+ \leq c \wedge y^- - x^+ \leq c \\
 x \leq c &\rightsquigarrow x^+ - x^- \leq 2c \\
 x \geq c &\rightsquigarrow x^- - x^+ \leq -2c
 \end{aligned}$$

Figure: Encoding rules for translating Program Invariants into Octagonal Constraints.